Hardware Ray Tracing for Bounded Decision-Path Scoring: An OptiX Backend Primitive

GAFIME Project
Technical Disclosure Draft

July 2026

Abstract

Tree ensembles and tree-derived decision-path features are among the most effective tools for tabular learning, but large-scale candidate scoring remains limited by candidate-row predicate evaluation, intermediate membership materialization, and reduction of binary indicators against a target. This paper describes an experimental CUDA backend primitive that maps bounded, shallow gradient-boosted decision-tree (GBDT) path regions onto NVIDIA hardware ray tracing using OptiX. A decision path over one to three numerical features is represented as an axis-aligned region, a tabular row is represented as a point, and path membership becomes a point-containment query. The low-level ABI keeps path descriptors compact and returns one result row per path with metric columns. The public Gafime Python adapter now uses this compact score ABI for a validated, complete unary Pearson/R^2 plan; unsupported plans retain explicit membership expansion. The measurements here still characterize the backend primitive, not end-to-end GBDT training, inference, or public-API acceleration.

The main performance contribution is not faster materialization of a dense row-by-path membership matrix. Instead, traversal results are reduced on device into compact per-path statistics and exported through the experimental result ABI. Traversal is hybrid: OptiX performs BVH traversal and intersection work, while programmable CUDA code performs exact guards, accumulation, and metric finalization inside a conservative supported geometry domain. For tree-leaf-like batches whose regions are finite, bounded, two-dimensional, and non-overlapping within each group, a first-hit traversal mode terminates after the first exact in-region hit and fails closed if the non-overlap proof is unavailable. The current implementation maps binary32 values to an exactly-representable ordered-integer lattice, constructs conservative custom primitive AABBs on that lattice, and evaluates the original fp32 predicates in the programmable intersection function. Functional tests on an RTX 4060 Laptop GPU establish membership and score parity across narrow normal spans, signed-zero and adjacent-ULP boundaries, translated ranges, and 1D/2D/3D paths; subnormal matrix values and thresholds remain outside the admitted domain. A fresh current-path checkpoint records 0.886494 ms warm p50 for first-hit RT and 39.374586 ms for the existing exhaustive SM fallback at 65,536 rows and 8,192 paths, with 4.65661×10^{-10} maximum error. The SM path is not a matched partition index, and the current custom-primitive `optixLaunch` is 2.31× slower than the retained triangle capture. The implementation therefore establishes a correctness-hardened systems primitive and a bounded performance checkpoint, not an RT-core speedup over a matched structure-aware CUDA baseline.

Keywords: GBDT systems; RT cores; OptiX; candidate scoring; fused reduction; tabular learning.
Artifact availability. The implementation, correctness tests, reproduction commands, retained profiler capture, timing transcription, and SHA-256 manifest are versioned in the GAFIME repository. A compact current-path checkpoint and source hashes are versioned; its raw 30 MiB profiler

report remains local. The reproduction note separates current custom-primitive evidence from historical triangle-prototype evidence.

1 Introduction

Gradient boosting decision trees are a standard workhorse for tabular machine learning. Friedman formulated gradient boosting as greedy function approximation [1]; XGBoost [2], LightGBM [3], and CatBoost [4] showed that practical performance depends as much on systems decisions as on statistical modeling. Cache-aware layout, sparsity handling, histogram construction, categorical encoding, and parallelism choices decide whether a tree system scales.

This work targets a related but distinct problem: scoring very large numbers of tree-derived decision-path candidates as explicit features. The intended GAFIME representation is a compact expression or path descriptor accompanied by computed metrics, rather than a candidate-expanded vector. The experimental CUDA ABI follows that representation. The public Python adapter now selects the compact score ABI for a complete, untruncated unary Pearson/R^2 family when a validated OptiX payload and finite representable geometry are available. Rust retains discovery and result ordering; unsupported shapes still materialize membership and feed the generic continuous engine. The scale problem is candidate-row work, not ordinary matrix multiplication.

A shallow decision path is a conjunction of threshold predicates. For example,

$$x_{f_0} > a \quad \wedge \quad x_{f_1} \leq b.$$

For one, two, or three participating features, this conjunction is naturally an interval, rectangle, or box. Hardware ray tracing units are specialized to traverse spatial acceleration structures and answer geometric intersection queries at high throughput. NVIDIA OPTIX provides a programmable ray tracing system for GPUs [15], and RTX-class GPUs include dedicated ray traversal and intersection hardware [17]. This suggests the following mapping:

$$\text{row} \longrightarrow \text{point}, \qquad \text{decision path} \longrightarrow \text{bounded region}.$$

The mapping is useful only if three engineering constraints hold. First, the geometric result must preserve the original predicate semantics, including strict and non-strict inequalities. Second, the output must remain compact; a fast path that writes a dense membership matrix moves the bottleneck rather than removing it. Third, backend ownership must remain explicit. In the target architecture Rust owns candidate semantics, scheduling, and fallback policy; CUDA executes only a validated request. The benchmark used here calls the low-level C ABI directly; separate public eager and compiled parity tests cover the narrow Rust-owned compact route.

This paper is a technical disclosure of the GAFIME v1 CUDA RT-core implementation. It makes the following contributions:

1. It formalizes a conservative ordered-binary32 lattice mapping for finite, low-dimensional GBDT path regions and an exact semantic guard.

2. It describes an experimental CUDA C ABI that scores decision paths through OPTIX and exports one compact row per path with metric columns.

3. It introduces a fail-closed first-hit mode for non-overlapping tree-leaf-like regions, together with the equivalence condition that makes first-hit traversal semantically valid.

4. It preserves a provenance-limited performance record for an earlier triangle prototype, with effective-ray and membership-equivalent rates kept separate and explicitly not transferred to the current implementation.

5. It reports a fresh current custom-primitive checkpoint against the real exhaustive SM fallback while identifying the missing matched partition-index comparison.

2 Background and Related Work

2.1 GBDT Systems

XGBoost combines gradient tree boosting with a sparsity-aware split finder, weighted quantile sketching, and cache-conscious execution [2]. LightGBM improves histogram-based training through gradient-based one-side sampling and exclusive feature bundling [3]. CatBoost addresses target leakage and categorical features through ordered boosting and specialized categorical handling [4]. GPU-oriented tree systems such as ThunderGBM [5] and GPU histogram training work [6] show that tree workloads can benefit from GPU parallelism when memory access, histogram conflicts, and algorithm structure are matched to the device.

The work here is not a replacement for these training systems. It focuses on candidate scoring for decision-path-derived features. That distinction matters: training chooses splits and updates an ensemble, while Gafime evaluates a large candidate set and keeps compact scores. The relevant unit of work is a membership-equivalent candidate-row evaluation rather than a tree-training iteration.

2.2 Tree Inference and Compilers

Tree deployment systems such as Treelite [7] and optimized forest inference paths such as NVIDIA FIL [8] target fast prediction for already-trained forests. Compiler approaches for tree inference, including Treebeard [9], lower tree traversal into optimized code for a selected target. These systems optimize inference of model outputs. Gafime instead scores candidate path features against a target and therefore needs compact reductions over path membership rather than final ensemble prediction alone.

2.3 Ray Tracing Systems

OptiX is a general-purpose programmable ray tracing engine [15]. Dedicated RTX hardware exposes fixed-function traversal and intersection acceleration [17]. The current design uses acceleration-structure traversal but supplies a programmable custom-primitive intersection function; it does not claim the fixed-function triangle-intersection path. GPU ray traversal work has studied how traversal efficiency depends on scheduling, memory locality, divergence, and scene structure [18, 19]. Embree [20] demonstrated that carefully engineered ray traversal kernels can be useful reusable systems infrastructure on CPUs. This paper applies ray traversal to a tabular candidate-scoring problem: not rendering, but evaluating whether row-points belong to many decision-path regions.

The closest methodological precedent is non-rendering use of RT hardware. Zellmann et al. map scientific point-containment queries to zero-length rays [10]. RTIndeX maps point and range lookups to GPU ray tracing [11], RTXRMQ maps range-minimum queries to closest-hit geometry [12], and RTScan maps conjunctive database scans into balanced 3D ray-tracing jobs [13]. These systems establish that a geometric reformulation can change both hardware use and algorithmic work. They also motivate a skeptical baseline: RasterScan retains a related grid data model

but replaces BVH traversal and geometric intersection with rasterization and arithmetic comparisons, outperforming its RT-based database-indexing baselines in that domain [14]. That result is not a direct measurement of decision-path scoring, but it shows why comparisons against equally structure-aware non-RT algorithms are required.

3 Decision Paths as Regions

Let $X \in \mathbb{R}^{n \times d}$ be a tabular matrix with n rows and d features, and let $y \in \mathbb{R}^n$ be a target. A path candidate p is a conjunction of threshold terms:

$$p = \{(f_\ell, s_\ell, t_\ell)\}_{\ell=1}^{L},$$

where f_ℓ is a feature id, $s_\ell \in \{\leq, >\}$ is the comparison, and t_ℓ is a threshold. Membership for row i is

$$z_{p,i} = \begin{cases} 1, & \text{if all terms in } p \text{ hold for row } i, \\ 0, & \text{otherwise.} \end{cases}$$

Definition 1 (Representable RT path)**.** *A path batch is representable by the current RT backend when every threshold is finite and non-subnormal, and every value in the resident feature matrix is finite and either zero or non-subnormal. The implementation stores matrix-wide finiteness and RT-representability bits, so a NaN, infinity, or subnormal in an unreferenced column still makes RT unsupported. A membership batch uses at most three feature axes in total; compact scoring may split paths into internal groups, each using at most three axes. Every admitted path uses the same custom-primitive representation. Optional membership calls use the exact SM implementation when representability or queried RT-core capability is unavailable. Compact scoring rejects a matrix containing any non-finite feature because its current SM bitset cannot encode the required missing-row exclusion; other finite-matrix RT failures may fall back. Require-RT and first-hit calls always fail closed.*

Each term tightens a lower or upper bound along a feature axis. For a path over unique feature axes $(u_1, \ldots, u_k)$, $k \leq 3$, the conjunction maps to

$$B_p = I_{p,1} \times \cdots \times I_{p,k},$$

where $I_{p,j}$ may be lower-open or upper-closed depending on the original predicate. A row maps to

$$q_i = (X_{i,u_1}, \ldots, X_{i,u_k}).$$

The desired semantic test is $z_{p,i} = 1$ if and only if $q_i \in B_p$ under the same open/closed comparison rules as the original path.

3.1 Ordered Binary32 Acceleration Lattice

Let $c(v)$ be the binary32 bit pattern of v, except that both signed zeros are canonicalized to the all-zero pattern. Define an unsigned order key

$$k(v) = \begin{cases} \sim c(v), & \text{if the sign bit is set,} \\ c(v) \oplus \texttt{0x80000000}, & \text{otherwise,} \end{cases}$$

and the bucket coordinate

$$b(v) = \left\lfloor \frac{k(v)}{2^9} \right\rfloor.$$

$$\begin{aligned} \text{GBDT path} &: x_{f_0} > a \ \wedge \ x_{f_1} \le b \\ \text{Region} &: (a, \infty) \times (-\infty, b] \\ \text{Row} &: q_i = (X_{i,f_0}, X_{i,f_1}) \\ \text{Score} &: \rho(z_p, y) \text{ or } R^2(z_p, y) \end{aligned}$$

candidate descriptors → RT traversal → on-device counts/sums → compact result rows.

Figure 1: Decision-path scoring as point-in-region traversal followed by compact on-device reduction. The experimental compact-score ABI does not export a dense row-by-path matrix.

For every admitted finite binary32 value, $b(v) < 2^{23}$, so converting the bucket integer back to binary32 is exact. The key is monotone in numerical order after signed-zero canonicalization; discarding low bits preserves monotonicity while deliberately allowing several adjacent values to share one bucket.

For a semantic interval $(\ell, h]$, the host constructs the acceleration interval

$$[b(\ell) - 1, \ b(h) + 1].$$

Unused axes use the finite extrema $-$`FLT_MAX` and `FLT_MAX`. A 1D/2D/3D path is represented by one custom AABB whose first two coordinates use these expanded bucket intervals and whose third coordinate is $[-\frac{1}{2}, \frac{1}{2}]$. Ray generation emits $(b(x), b(y), -2)$ with direction $(0, 0, 1)$ and trace range $[0, 4]$. For 3D paths, the original third feature remains in the semantic point but is not used for broad-phase culling.

Lemma 1 (Conservative candidate generation)**.** *For any admitted row point q and path region B_p, if $q \in B_p$ under the original open-lower/closed-upper predicates, then the encoded ray intersects the custom AABB associated with B_p on a conforming OptiX implementation.*

Proof. For each accelerated axis, monotonicity gives $b(\ell) \le b(q) \le b(h)$ whenever $\ell < q \le h$. The encoded coordinate is therefore inside the deliberately expanded interval. The z ray crosses $[-\frac{1}{2}, \frac{1}{2}]$ within its trace range. The custom AABB thus completely encloses every encoded point that can satisfy the semantic region, which is the required OptiX custom-primitive enclosure contract [16]. □

The converse is intentionally not claimed: bucket collisions and one-unit expansion create broad-phase false positives. Before geometry construction, the host reduces repeated predicates on each axis to the strongest open lower bound and strongest closed upper bound. Because the original terms are a conjunction, this interval has exactly the same accepted set. The custom intersection program reloads the original coordinates, evaluates that collapsed conjunction, and reports a hit only on exact acceptance. Consequently broad-phase false positives cannot become membership false positives, while the lemma excludes semantic false negatives within the admitted domain.

4 Scoring Without Dense Membership

The compact score path currently supports Pearson correlation and R^2 for decision-path indicators. Non-finite targets are excluded from every statistic. Let $V = \{i \mid y_i \text{ is finite}\}$, $n_V = |V|$, and $\bar{y} = n_V^{-1} \sum_{i \in V} y_i$ when $n_V > 0$. For path p, define

$$S_x = \sum_{i \in V} z_{p,i}, \qquad C_{yy} = \sum_{i \in V} (y_i - \bar{y})^2, \qquad C_{xy} = \sum_{i \in V} z_{p,i}(y_i - \bar{y}).$$

The remaining centered indicator term is

$$C_{xx} = S_x - \frac{S_x^2}{n_V}.$$

The Pearson score is

$$\rho_p = \begin{cases} \operatorname{clip}\left(C_{xy}/\sqrt{C_{xx}C_{yy}}, -1, 1\right), & C_{xx}C_{yy} > 0, \\ 0, & \text{otherwise}, \end{cases}$$

and $R_p^2 = \operatorname{clip}(\rho_p^2, 0, 1)$. When $n_V = 0$, the implementation returns zero for both metrics.

This formula shows why dense membership export is unnecessary. The score needs only per-path counts and centered target sums plus target-wide statistics. Therefore, the backend can evaluate membership transiently, reduce on device, and return one compact row per candidate with metric columns.

5 System Design

5.1 Ownership Boundary

The intended GAFIME v1 architecture uses a hard ownership split. Python exposes the public API; Rust owns validation, feature planning, scheduling, backend selection, fallback policy, and report ownership; CUDA owns the CUDA-specific execution boundary. The experimental ABI implements the backend half of this contract. The current public adapter invokes it only for the validated complete-unary Pearson/R^2 shape and keeps every unsupported case on the established explicit fallback.

- An approved Rust caller passes compact, validated decision-path descriptors and an explicit RT policy through the GPU C ABI.
- CUDA validates RT representability, builds geometry, launches OptiX, and writes compact result rows.
- CUDA does not discover candidate paths, mutate scheduler policy, or infer Python behavior. A require-RT request fails explicitly if unavailable.
- First-hit requests return unsupported when their finite, bounded, two-dimensional, non-overlap proof cannot be established.

The explicit per-call policy is `DecisionPathRtPolicy::RequireRt`, encoded as `GAFIME_DECISION_PATH_FLAG_REQUIRE_RT` in the batch. It controls fallback only: an unavailable or unrepresentable RT request returns unsupported instead of executing the SM comparator. By contrast, the CUDA payload reads two process-global experimental test and profiling selectors:

- `GAFIME_CUDA_DECISION_PATH_RT`,
- `GAFIME_CUDA_DECISION_PATH_RT_SCORE`.

They are not a thread-local or per-call public policy ABI. The benchmark selects `firsthit` through the process environment and independently sets the per-call require-RT flag so an accidental SM fallback cannot produce a timing.

5.2 CUDA Source Split

The implementation keeps RT code out of the generic continuous CUDA kernels:

- `src/cuda/rt_kernels.cu` contains OptiX device programs, RT-specific CUDA kernels, point packing, exact hit filters, and direct-score accumulation helpers.
- `src/cuda/rt_launcher.cu` owns host-side RT planning, finite-box validation, OptiX pipeline setup, GAS/IAS construction, cached workspaces, grouped execution, and explicit comparator dispatch.
- `src/cuda/launcher.cu` remains the public C ABI bridge for decision-path RT symbols.

This source layout is part of the architectural contract. RT traversal logic is large enough to deserve explicit ownership; hiding it inside the generic CUDA score path would make later backend maintenance harder.

5.3 Grouped Instanced Launch

Candidate batches may contain many feature pairs. A single custom-primitive GAS cannot interpret point coordinates for all feature-pair coordinate systems at once. The CUDA launcher groups paths by compatible feature axes, builds one custom AABB GAS per group, and wraps group GAS handles in one instance acceleration structure (IAS). Ray generation then launches

$$\text{launch shape} = (n,\ g,\ 1),$$

where g is the number of RT groups. The instance id selects the group-local path table, and a scatter map restores original path order before result export. This gives OptiX a large batched launch without moving candidate discovery into the backend.

5.4 Score Modes

Bitset mode. OptiX writes an idempotent device bitset, and a CUDA kernel reduces the bitset with the target vector. This is the conservative parity path because it avoids floating accumulation order in the traversal program.

Direct mode. The any-hit program updates per-path counts and centered target sums directly. Target mean and centered variance are computed in double precision, and the centered per-hit sum uses double-precision atomics. This retains a path–row bitset for idempotent custom-primitive callbacks and removes the separate reduction scan, but uses floating atomics for some statistics, so it is opt-in and documented with tolerance.

First-hit mode. For finite, bounded, two-dimensional groups where CUDA proves that boxes do not overlap, the any-hit program accepts the first exact hit and terminates traversal. This matches tree-leaf assignment: a row belongs to at most one leaf region in a partition.

Proposition 1 (First-hit equivalence)**.** *Let $\mathcal{B} = \{B_1, \ldots, B_m\}$ be a group of bounded two-dimensional decision regions such that the interiors are pairwise disjoint and the exact open/closed predicate guard assigns every boundary point to at most one region. Represent every region with the conservative custom AABB above. For any row point q, first-hit traversal with the exact guard returns the same membership vector as evaluating all path predicates and selecting the unique inside region, if one exists.*

Proof. If q is outside all regions, no exact guard accepts a candidate, so all membership values are zero. If q is inside one region B_j, conservative candidate generation supplies a candidate for B_j, while disjointness and the boundary ownership guard imply no other region can also accept q. Terminating after the first accepted hit therefore records B_j and cannot suppress another valid region. The recorded membership is identical to exhaustive predicate evaluation. □

First-hit mode is fail-closed. If the finite, bounded, 2D, and non-overlapping checks are not satisfied, the experimental selector returns unsupported rather than changing semantics or silently routing that call to the SM comparator. The independent require-RT ABI flag also forbids fallback.

RT support is also a build-time distribution choice. The default CUDA payload is RT-disabled and has no OptiX dependency. CMake build modes `off`, `on`, and `both` produce an RT-free payload, an RT-enabled payload, or distinct copies of each. Every measurement in this paper uses the explicitly RT-enabled `libgafime_cuda_v1_rt.so`; it is not evidence about the standard RT-off artifact. The optional RT wheel is a separately selected GitHub Actions artifact, not a claimed PyPI release.

RT availability is queried from the OptiX context through `OPTIX_DEVICE_PROPERTY_RTCORE_VERSION`. A zero value is unsupported; CUDA architecture-family names are not used as a substitute capability proof [16].

6 Numerical Policy

The correctness policy separates Boolean membership from floating score accumulation. For every candidate intersection, the programmable guard implements the GAFIME predicate model exactly: `LE` means $x \leq t$, `GT` means $x > t$, and NaNs are not silently reinterpreted. The current upload records RT representability for the entire feature matrix, not only columns referenced by a path; any non-finite or subnormal feature value makes the RT path unsupported. Non-finite target values do not make RT unsupported, but those rows are omitted from all Pearson/R^2 statistics as defined above. Functional regression tests assert that subnormal matrix values fail required RT and use the exact SM path only when fallback is permitted. Minimum-normal values, normal spans below the former 2^{-60} boundary, translated adjacent-ULP cells, signed-zero crossings, high-magnitude cells, and explicit 3D boxes now execute through required RT and match the CPU membership oracle. The ordered lattice proof covers candidate generation; the original-value intersection guard remains the authority for membership.

Path ids, result row ordering, valid-target counts, and direct-mode hit counts remain integral under the RT row limit of `UINT32_MAX`. Target mean, centered target variance, bitset covariance reduction, and direct centered-sum atomics use double precision; the public Pearson and R^2 outputs are rounded to binary32. Centering avoids cancellation for targets with a large common offset, which is covered by a regression case near 10^8. Direct-mode values may still differ because atomic order is not deterministic. The historical first-hit parity run had maximum absolute score difference 1.19209×10^{-7} against the CPU reference, below the documented spike tolerance of 10^{-4}.

7 Evaluation

7.1 Current Correctness Validation

The ordered-float custom-primitive payload was built from the current source with CUDA 13.3 and OptiX SDK 9.1, then exercised through the Rust C-ABI wrapper on the RTX 4060 Laptop GPU. The serial `gafime-gpu-sys` run completed 61 crate tests and eight RT numeric-domain integration

tests without failure. The integration cases require OptiX execution rather than accepting SM fallback and cover: normal spans immediately below, at, and above the former cutoff; extreme aspect ratios; minimum-normal values; signed zero; adjacent ULPs near one and at high magnitude; translated open/closed cells; cache replacement; 3D membership; three-axis exact-pair grouping; and large-offset target score parity across bitset, direct, and first-hit modes. A separate case verifies that a subnormal matrix fails required RT and remains correct only through explicit SM fallback. These are functional tests, not performance measurements.

7.2 Hardware and Workloads

The archived measurements in this disclosure were collected from the superseded triangle prototype on a laptop system with an AMD Ryzen AI 9 HX 370 class CPU and an NVIDIA GeForce RTX 4060 Laptop GPU (Ada, sm_89, approximately 7.63 GiB reported global memory), CUDA 13.3, and OptiX 7.5 headers. The exact benchmark sources are:

`tests/gpu/cuda_rt_membership_scale_bench.cpp`
`tests/release_measure/perf_05_cuda_rt_firsthit_scale.py`

Complete build, execution, profiler, and PDF commands are recorded in:

`docs/rt-gbdt-paper-repro.md`

The historical implementation and harness files against which those archived claims were reviewed are pinned by:

`docs/evidence/rt-gbdt-paper-source-evidence.sha256`

Its own SHA-256 is:

`1eb2db483b5ad2881e99dbaf711af192ad6bfbfb294db443d3c5e6745f2ed429`

The preserved timing transcription is:

`docs/evidence/rt-firsthit-sm89-timing.txt`

Its SHA-256 is:

`8fe2b167ecf69597cf68d34137b354fe659d18617e2b5497737157d18955c230`

The reproduction note gives exact commands for validating the historical source/evidence identity and for replaying the configurations against the current source. Neither raw benchmark stdout nor the original executable hash was retained. The transcript hash therefore proves only the identity of the preserved transcription; it does not authenticate the measurement or a byte-identical capture binary. A fresh current-path checkpoint is recorded separately below. Its raw profiler report is not versioned, so the historical capture remains the only archived report suitable for independent re-digestion.

The synthetic workloads use bounded 2D boxes over feature-pair groups. The partitioned-grid mode creates non-overlapping boxes within each group, modeling the tree-leaf invariant required by first-hit traversal. We report all-pairs-equivalent work as

$$W_{\text{eq}} = n \times m,$$

where n is row count and m is path count. First-hit traversal does not execute $n \times m$ explicit predicate tests. Its actual launch contains $n \times g$ rays for g compatible geometry groups, and BVH traversal prunes the path set. Dividing $n \times g$ by the full timed score call produces an end-to-end effective ray rate; it includes launch and programmable CUDA work and is not an isolated RT-core counter. We report it beside W_{eq}/t.

7.3 Current Custom-Primitive Checkpoint

The current first-hit custom-primitive path was measured in five fresh processes at 65,536 rows, 8,192 paths, and eight compatible groups. Each process recorded one first call separately and used eight resident calls for its warm p50. The median of process p50s was 0.886494 ms for RT and 39.374586 ms for the existing exhaustive SM fallback, an observed ratio of 44.416×. Both paths matched the exact partition oracle within 4.65661×10^{-10}. The RT value corresponds to 605.611 G membership-equivalent candidate-row evaluations/s and 0.591 G end-to-end effective rays/s. Median first-call latency was 52.562210 ms for RT and 30.308569 ms for SM.

This is a comparison against the real fallback, not a causal attribution to RT hardware: the SM comparator evaluates every row–path pair and does not exploit the known partition index. PerfDigest analysis of a full-set Nsight capture also found the current custom-primitive `optixLaunch` at 455.904 μs, versus 196.992 μs in the retained triangle capture. DRAM peak activity rose from 10.878% to 27.965%, L2 hit rate fell from 96.864% to 88.389%, and registers remained 72 per thread. The custom representation closes numeric domain defects but leaves a concrete traversal and memory-locality regression to optimize. The compact evidence record is versioned in `docs/evidence/rt-firsthit-custom-sm89-checkpoint.txt`; the current raw report is not.

According to the preserved development transcript, each score run made six calls after one resident matrix upload. The first timed call includes OptiX program initialization, geometry construction, and cache population. The next five calls reuse resident state; the reported warm number is the observed median sample, not a best-of-N value. The benchmark validates the worst parity error across the cold call and all five warm samples. Because the primary stdout and executable were not retained, these warm values are provisional observations and must not be read as publication-grade steady-state evidence or first-use latency.

7.4 Compact Output and Score Modes

Materializing an `f32` path-major membership matrix for $1{,}048{,}576 \times 512$ requires 2.00 GiB. A byte mask requires 512 MiB, the conservative device bitset requires 64 MiB, and the final two-metric result requires only 4 KiB. Compact reduction is therefore part of the algorithm, not an output-format detail.

The current RT bitset path retains a deterministic idempotent mask. Direct mode removes the mask scan and accumulates traversal statistics with floating atomics; it is experimental and uses the documented 10^{-4} tolerance. The preserved timing transcription below is limited to first-hit mode; this paper does not attach a timing claim to the other score modes.

7.5 First-Hit Partitioned Score

The most promising historical observation appeared when the regions satisfied the tree-leaf-like non-overlap condition. Table 1 preserves both end-to-end effective ray rate and all-pairs-equivalent work rate for eight geometry groups; the values are provisional for the provenance reasons above.

Table 1: Provisional first-hit development transcript at 262,144 rows. Warm p50 uses five resident samples. Raw stdout and the executable hash were not retained. The small case used an exhaustive CPU reference; the large case used the exact partition oracle.

Paths	Paths/group	First ms	Warm ms	G eff. ray/s	T eq/s	Max abs
8,192	1,024	56.109	0.886	2.367	2.424	1.19209×10^{-7}
1,048,576	131,072	467.746	20.180	0.104	13.621	5.58794×10^{-9}

The hash-bound transcription for the parity-covered case records:

first RT call	56.109 ms,
resident warm p50	0.886 ms (2.424 T eq/s),
maximum absolute difference	1.19209×10^{-7}.

The large case no longer skips correctness. Its oracle exploits the known partition structure: for each row and group it identifies the unique cell using the same floating open-lower/closed-upper boundaries, accumulates that path's statistics, and finalizes every path. This is $O(ng + m)$ rather than an exhaustive $O(nm)$ scan. The benchmark also rejects non-finite scores and checks result row count, order, family, rank, and candidate id on every repetition.

7.6 Profiler Evidence

A separate Nsight Compute 2026.2.1 full-set capture profiled one triangle-path 65,536 × 8,192 first-hit call. Full replay perturbs end-to-end timing, so these durations are used only to identify hot units. PerfDigest reduced the report without copying the raw counter stream into the review context. The exact 31,848,275-byte capture and its digest are:

`docs/evidence/rt-firsthit-sm89-65536x8192-final.ncu-rep`

`SHA-256 5461bf86495d9a12666891bba2f334ecea8b16b3c8cb806168a557101a52c331`

The reproduction note pins the exact PerfDigest calls; the digest is not taken from the similarly named earlier review capture. The cold profiled call contained an `optixLaunch` at 196.992 μs, grouped point packing at 25.088 μs, and score scatter/finalization at 4.480 μs. Eight group GAS builds were approximately 156 μs each and the IAS build was 49.952 μs; those builds belong to first-use setup.

For `optixLaunch`, the available counters report 24.932% compute-pipe peak, 10.878% DRAM peak, 54.223% achieved occupancy, 53.408% L1 hit, and 96.864% L2 hit, with 72 registers/thread. These measurements show that the profiled launch is longer than point packing and score scatter. That historical launch included fixed-function triangle intersection, traversal, exact programmable guards, and atomic score accumulation, so the counters do not isolate traversal or measure RT-core saturation. Neither may be inferred from SM compute, cache, or branch metrics, and none of these values characterizes the current custom intersection program.

7.7 Interpretation

The historical first-hit result was large because the workload matched both the BVH algorithm and the traversal hardware:

- path candidates are bounded regions, not arbitrary branch programs;
- the partitioned-grid invariant lets each row-group ray terminate after the first exact hit;
- grouped instancing creates a large launch rather than many tiny launches;
- score output is compact and does not copy dense membership to host;
- resident sessions reuse geometry, packed points, target statistics, and scratch buffers when inputs are unchanged.

Dense overlapping boxes do not show the same behavior. The provisional timing transcription therefore cannot be interpreted as a general RT-core speedup over exhaustive CUDA. The first-hit method changes the algorithm from all-pairs testing to BVH search with at most one accepted hit per row-group ray. A fair attribution study still needs a CUDA baseline that exploits the same partition structure without OptiX.

8 Limitations

The implementation is intentionally narrow. The first-hit path currently applies only to finite, bounded, non-overlapping 2D groups. The compact RT decision-path score ABI supports Pearson and R^2 in this spike; MI and Spearman require additional parity work. CUDA/OptiX is the only RT backend. ROCm and Metal do not expose this RT-core path through the current Gafime ABI.

Direct and first-hit modes currently retain a duplicate-safety path–row bitset. Its storage is $m\lceil n/32 \rceil$ 32-bit words: the historical $262{,}144 \times 1{,}048{,}576$ shape would require 32 GiB before other RT workspace allocations. The superseded triangle prototype did not carry this current allocation contract, so its million-path timing cannot be transferred to the present implementation. Scaling this design requires a bounded duplicate suppression scheme or a proof that safely removes the bitset.

The public Python decision-path workflow invokes the compact score ABI only for the exact complete-unary Pearson/R^2 shape. Mixed or higher arity, truncated families, non-finite geometry, graph execution, significance, and unsupported payloads retain generic membership expansion. This paper still makes no end-to-end Gafime speedup claim because the current benchmark bypasses public planning and no matched partition-index baseline exists. The default CUDA distribution is also RT-disabled; users must select an RT-enabled payload explicitly.

Both evaluated workloads are synthetic partition grids. The large case is validated by a structure-aware CPU oracle rather than exhaustive all-pairs CPU evaluation. No production ensemble paths, training loop, inference service, or feature-mining search were benchmarked. The current SM comparator evaluates all row-path pairs and is not an algorithmically matched partition index, so it cannot isolate the contribution of traversal hardware or the programmable custom-intersection design.

The evaluation uses one Ada laptop GPU. Larger desktop RTX devices or different generations may shift the bottleneck toward acceleration-structure size, target-stat atomic pressure, memory bandwidth, or launch scheduling. Available Nsight Compute counters did not expose a direct RT-core saturation percentage. First-call latency is tens to hundreds of milliseconds because it includes program and acceleration-structure setup; resident warm timing applies only when unchanged geometry and buffers can be reused.

9 Future Work

Several pieces remain before this becomes a broader GBDT acceleration method:

1. broaden and benchmark the Rust-owned public compact route beyond its current complete-unary Pearson/R^2 contract without candidate-expanded row materialization;
2. evaluate paths extracted from real trained ensembles, not only synthetic partition grids;
3. compare against a CUDA partition index with the same first-hit algorithmic advantage, then separate build, query, memory, and energy costs;

4. add deterministic buffered first-hit statistics and compare them with direct floating atomics;
5. extend compact RT scoring to MI and Spearman only after estimator and significance parity are proven;
6. test Turing, Ampere, Ada, Hopper, and Blackwell devices and report acceleration-structure memory explicitly;
7. compare against production tree-inference and GPU-GBDT systems while preserving the distinction between candidate scoring, inference, and training.

10 Conclusion

This paper reports a systems mapping from bounded decision-path scoring to hardware ray tracing traversal and on-device fused reduction. The key insight is not that every tree operation is geometric. The useful subset is narrower: finite low-dimensional path regions can be scored as point-in-region tests, and tree-leaf-like non-overlap makes first-hit traversal semantically valid.

The implementation establishes a compact low-level ABI, a conservative ordered-binary32 candidate mapping, exact original-value guards, and fail-closed representability rules. A narrow complete-unary Pearson/R^2 route is integrated through the public eager and compiled APIs, while broader plans retain explicit fallback. The superseded triangle prototype recorded 2.424 T membership-equivalent evaluations/s at 2.367 G end-to-end effective rays/s on its parity-covered case, and 13.621 T/s at 0.104 G effective rays/s on its larger oracle-validated case. Those values are historical hypotheses, not performance results for the current custom-primitive implementation. The central supported conclusion is narrower: tree-like partition structure can be mapped to conservative BVH search and compact CUDA reduction without changing admitted membership semantics. It is not evidence that RT cores accelerate arbitrary GBDT computation.

References

[1] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. https://doi.org/10.1214/aos/1013203451.

[2] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. https://doi.org/10.1145/2939672.2939785.

[3] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 30, 2017. https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.

[4] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, 31, 2018. https://papers.nips.cc/paper/7898-catboost-unbiased-boosting-with-categorical-features.

[5] Zeyi Wen, Hanfeng Liu, Jiashuai Shi, Qinbin Li, Bingsheng He, and Jian Chen. ThunderGBM: Fast GBDTs and random forests on GPUs. *Journal of Machine Learning Research*, 21(108):1–5, 2020. https://www.jmlr.org/papers/v21/19-095.html.

[6] Huan Zhang, Si Si, and Cho-Jui Hsieh. GPU-acceleration for large-scale tree boosting. arXiv:1706.08359, 2017. https://arxiv.org/abs/1706.08359.

[7] Hyunsu Cho and Mu Li. Treelite: Toolbox for decision tree deployment. In *Systems for Machine Learning*, 2018. https://www.amazon.science/publications/treelite-toolbox-for-decision-tree-deployment.

[8] NVIDIA RAPIDS. Forest Inference Library documentation. https://docs.rapids.ai/api/cuml/stable/fil/.

[9] Ashwin Prasad, Sampath Rajendra, Kaushik Rajan, R. Govindarajan, and Uday Bondhugula. Treebeard: An Optimizing Compiler for Decision Tree Based ML Inference. In *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture*, 2022. https://doi.org/10.1109/MICRO56248.2022.00043.

[10] Stefan Zellmann, Daniel Seifried, Nate Morrical, Ingo Wald, Will Usher, Jamie A. P. Law-Smith, Stefanie Walch-Gassner, and Andre Hinkenjann. Point containment queries on ray tracing cores for AMR flow visualization. arXiv:2202.12020, 2022. https://arxiv.org/abs/2202.12020.

[11] Justus Henneberg and Felix Schuhknecht. RTIndeX: Exploiting hardware-accelerated GPU raytracing for database indexing. *Proceedings of the VLDB Endowment*, 16(13):4268–4281, 2023. https://doi.org/10.14778/3625054.3625063.

[12] Enzo Meneses, Cristobal A. Navarro, Hector Ferrada, and Felipe A. Quezada. Accelerating range minimum queries with ray tracing cores. arXiv:2306.03282, 2023. https://arxiv.org/abs/2306.03282.

[13] Yangming Lv, Kai Zhang, Ziming Wang, Xiaodong Zhang, Rubao Lee, Zhenying He, Yinan Jing, and X. Sean Wang. RTScan: Efficient scan with ray tracing cores. *Proceedings of the VLDB Endowment*, 17(6):1460–1472, 2024. https://doi.org/10.14778/3648160.3648183.

[14] Harish Doraiswamy and Jayant R. Haritsa. Raster is faster: Rethinking ray tracing in database indexing. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2026. https://www.vldb.org/cidrdb/papers/2026/p18-doraiswamy.pdf.

[15] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. OptiX: A general purpose ray tracing engine. *ACM Transactions on Graphics*, 29(4), 2010. https://doi.org/10.1145/1778765.1778803.

[16] NVIDIA. NVIDIA OptiX 9.0 Programming Guide and API Reference. 2025. https://raytracing-docs.nvidia.com/optix9/guide/index.html.

[17] NVIDIA. NVIDIA Turing GPU Architecture. White paper, 2018. https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf.

[18] Timo Aila and Samuli Laine. Understanding the efficiency of ray traversal on GPUs. In *Proceedings of High Performance Graphics*, 2009. https://users.aalto.fi/~ailat1/HPG2009/.

[19] Timo Aila, Samuli Laine, and Tero Karras. Understanding the efficiency of ray traversal on GPUs: Kepler and Fermi addendum. NVIDIA Technical Report NVR-2012-02, 2012. https://research.nvidia.com/publication/2012-06_understanding-efficiency-ray-traversal-gpus-kepler-and-fermi-addendum.

[20] Ingo Wald, Sven Woop, Carsten Benthin, Gregory S. Johnson, and Manfred Ernst. Embree: A kernel framework for efficient CPU ray tracing. *ACM Transactions on Graphics*, 33(4), 2014. https://doi.org/10.1145/2601097.2601199.